

7. 栈和队列

【本节目标】

1. 栈的概念及使用
2. 队列的概念及使用
3. 相关OJ题

1. 栈(Stack)

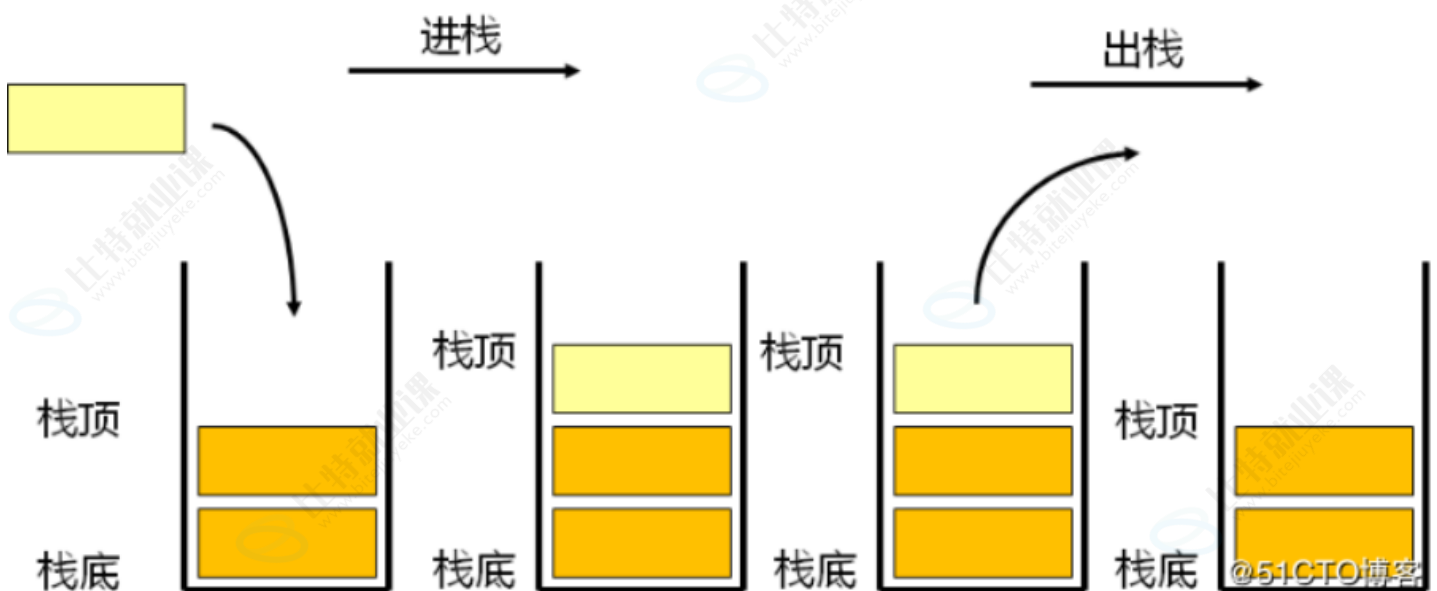
1.1 概念

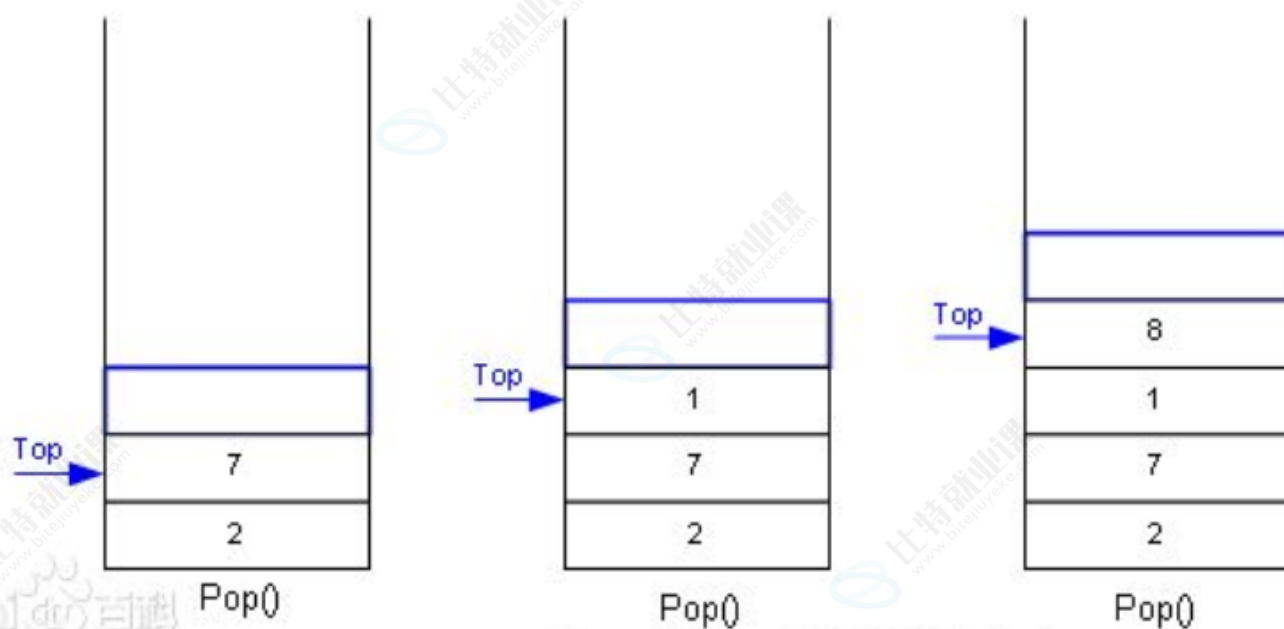
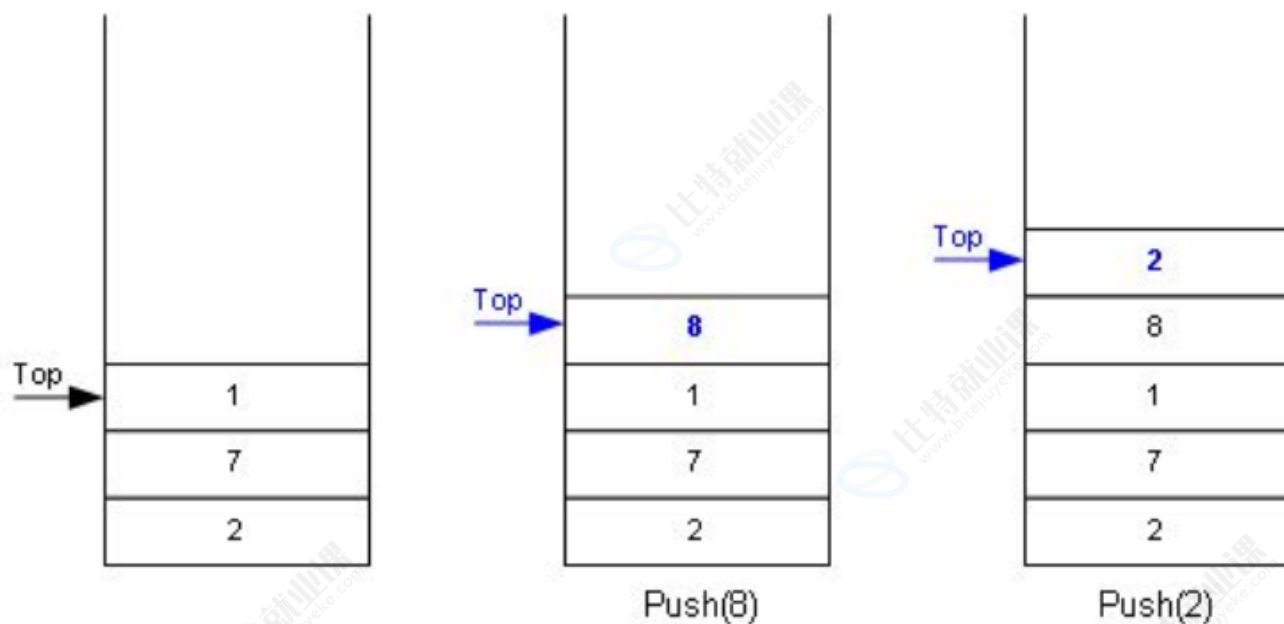
栈：一种特殊的线性表，其只允许在固定的一端进行插入和删除元素操作。进行数据插入和删除操作的一端称为栈顶，另一端称为栈底。栈中的数据元素遵守后进先出LIFO（Last In First Out）的原则。

压栈：栈的插入操作叫做进栈/压栈/入栈，**入数据在栈顶**。

出栈：栈的删除操作叫做出栈。**出数据在栈顶**。

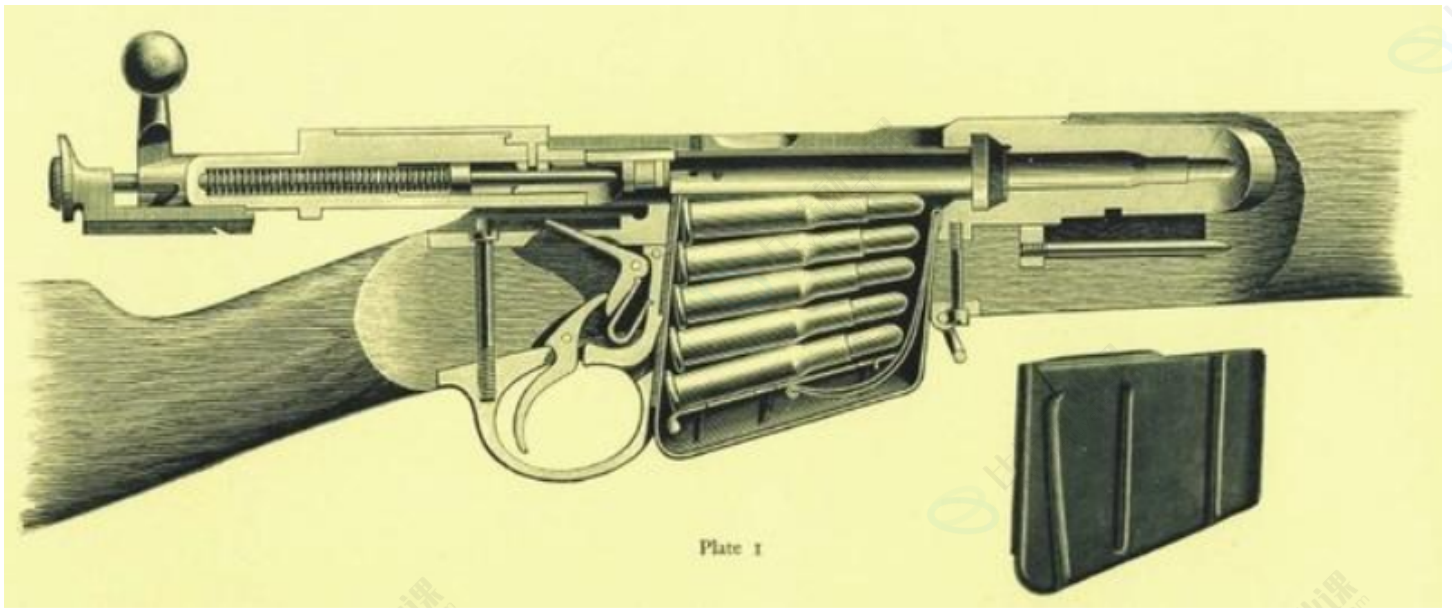
— 后进先出 (Last In First Out)





Stack的Push和Pop，遵循先进后出规则

栈在现实生活中的例子

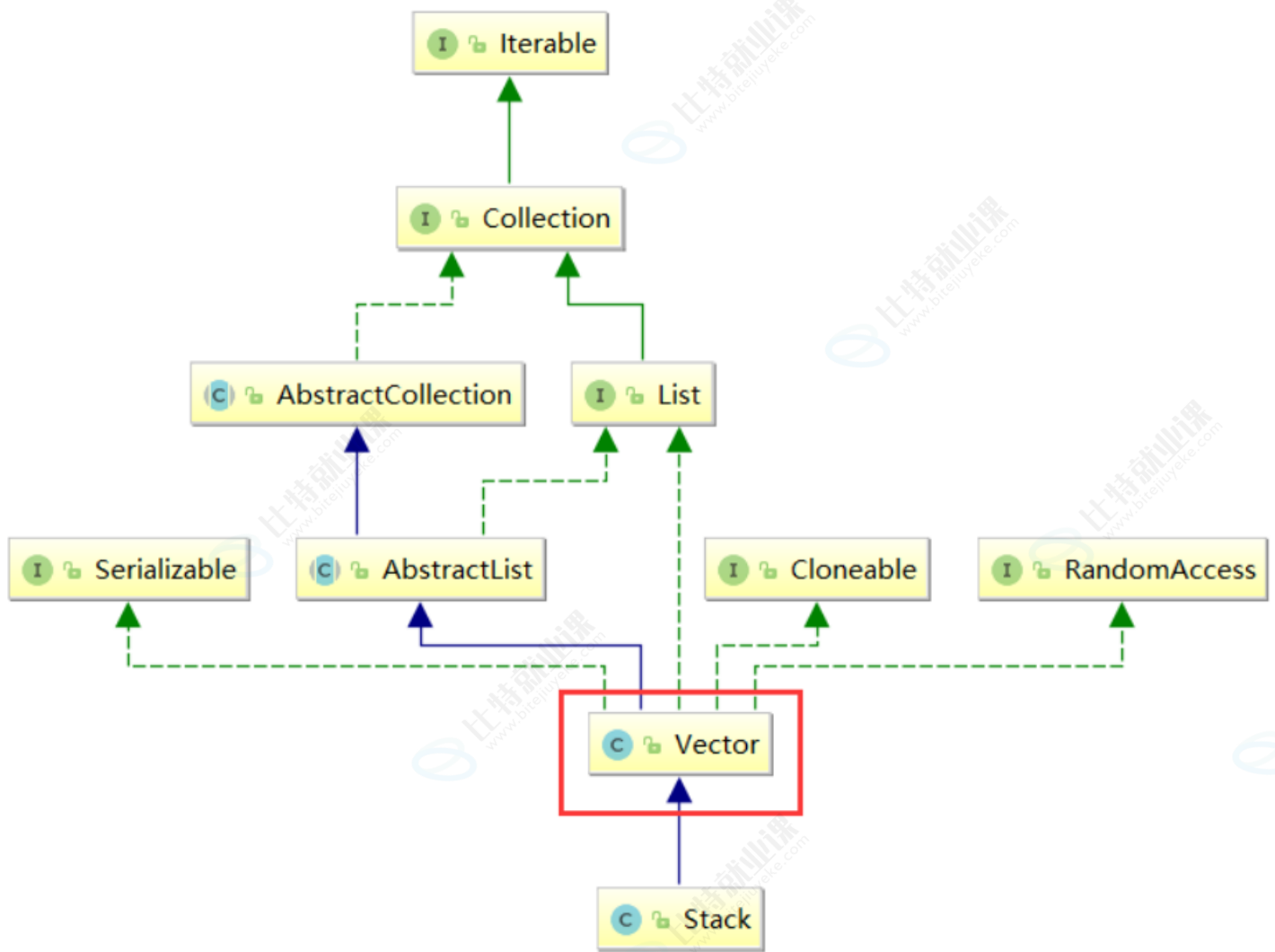


1.2 栈的使用

方法	功能
Stack()	构造一个空的栈
E push(E e)	将e入栈，并返回e
E pop()	将栈顶元素出栈并返回
E peek()	获取栈顶元素
int size()	获取栈中有效元素个数
boolean empty()	检测栈是否为空

```
1 public static void main(String[] args) {
2     Stack<Integer> s = new Stack();
3     s.push(1);
4     s.push(2);
5     s.push(3);
6     s.push(4);
7     System.out.println(s.size());    // 获取栈中有效元素个数---> 4
8     System.out.println(s.peek());    // 获取栈顶元素---> 4
9     s.pop();    // 4出栈，栈中剩余1 2 3，栈顶元素为3
10    System.out.println(s.pop());    // 3出栈，栈中剩余1 2 栈顶元素为3
11    if(s.empty()){
12        System.out.println("栈空");
13    }else{
14        System.out.println(s.size());
15    }
16 }
```

1.3 栈的模拟实现



从上图中可以看到，Stack继承了Vector，Vector和ArrayList类似，都是动态的顺序表，不同的是Vector是线程安全的。

```
1 public class MyStack {
2     int[] array;
3     int size;
4
5     public MyStack(){
6         array = new int[3];
7     }
8
9     public int push(int e){
10        ensureCapacity();
11        array[size++] = e;
12        return e;
13    }
14
15    public int pop(){
```

```

16         int e = peek();
17         size--;
18         return e;
19     }
20
21     public int peek(){
22         if(empty()){
23             throw new RuntimeException("栈为空，无法获取栈顶元素");
24         }
25
26         return array[size-1];
27     }
28
29     public int size(){
30         return size;
31     }
32
33     public boolean empty(){
34         return 0 == size;
35     }
36
37     private void ensureCapacity(){
38         if(size == array.length){
39             array = Arrays.copyOf(array, size*2);
40         }
41     }
42 }

```

1.4 栈的应用场景

1. 改变元素的序列

- 1 1. 若进栈序列为 1,2,3,4，进栈过程中可以出栈，则下列不可能的一个出栈序列是 ()
- 2 A: 1,4,3,2 B: 2,3,4,1 C: 3,1,4,2 D: 3,4,2,1
- 3
- 4 2. 一个栈的初始状态为空。现将元素1、2、3、4、5、A、B、C、D、E依次入栈，然后再依次出栈，则元素出栈的顺序是 ()。
- 5 A: 12345ABCDE B: EDCBA54321 C: ABCDE12345 D: 54321EDCBA

2. 将递归转化为循环

比如：逆序打印链表

```
1 // 递归方式
```

```

2 void printList(Node head){
3     if(null != head){
4         printList(head.next);
5         System.out.print(head.val + " ");
6     }
7 }
8
9 // 循环方式
10 void printList(Node head){
11     if(null == head){
12         return;
13     }
14
15     Stack<Node> s = new Stack<>();
16     // 将链表中的结点保存在栈中
17     Node cur = head;
18     while(null != cur){
19         s.push(cur);
20         cur = cur.next;
21     }
22
23     // 将栈中的元素出栈
24     while(!s.empty()){
25         System.out.print(s.pop().val + " ");
26     }
27 }

```

3. 括号匹配

4. 逆波兰表达式求值

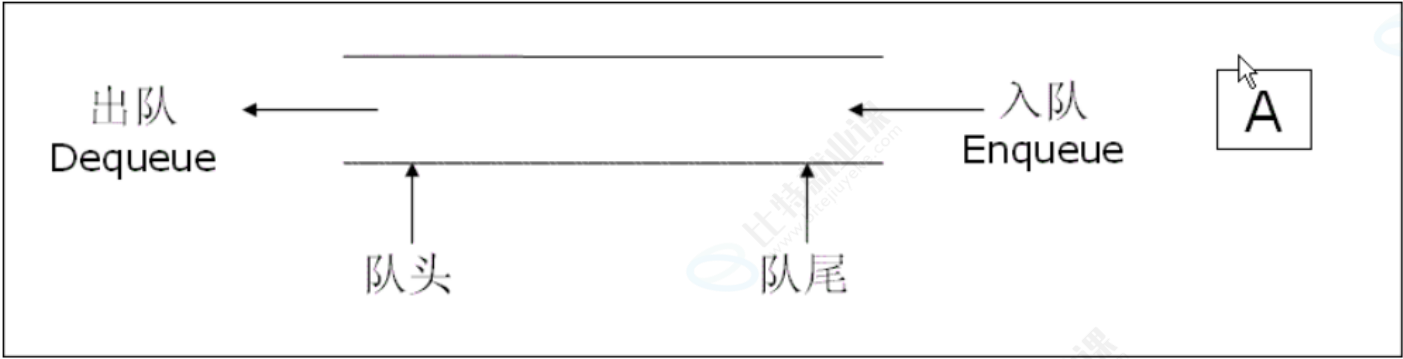
5. 出栈入栈次序匹配

6. 最小栈

2. 队列(Queue)

2.1 概念

队列：只允许在一端进行插入数据操作，在另一端进行删除数据操作的特殊线性表，队列具有先进先出FIFO(First In First Out)入队列：进行插入操作的一端称为**队尾 (Tail/Rear)** 出队列：进行删除操作的一端称为**队头 (Head/Front)**



2.2 队列的使用

在Java中，Queue是个接口，底层是通过链表实现的。



方法	功能
boolean offer(E e)	入队列
E poll()	出队列
peek()	获取队头元素
int size()	获取队列中有效元素个数
boolean isEmpty()	检测队列是否为空

注意：Queue是个接口，在实例化时必须实例化LinkedList的对象，因为LinkedList实现了Queue接口。

```
1 public static void main(String[] args) {
2     Queue<Integer> q = new LinkedList<>();
3     q.offer(1);
4     q.offer(2);
}
```



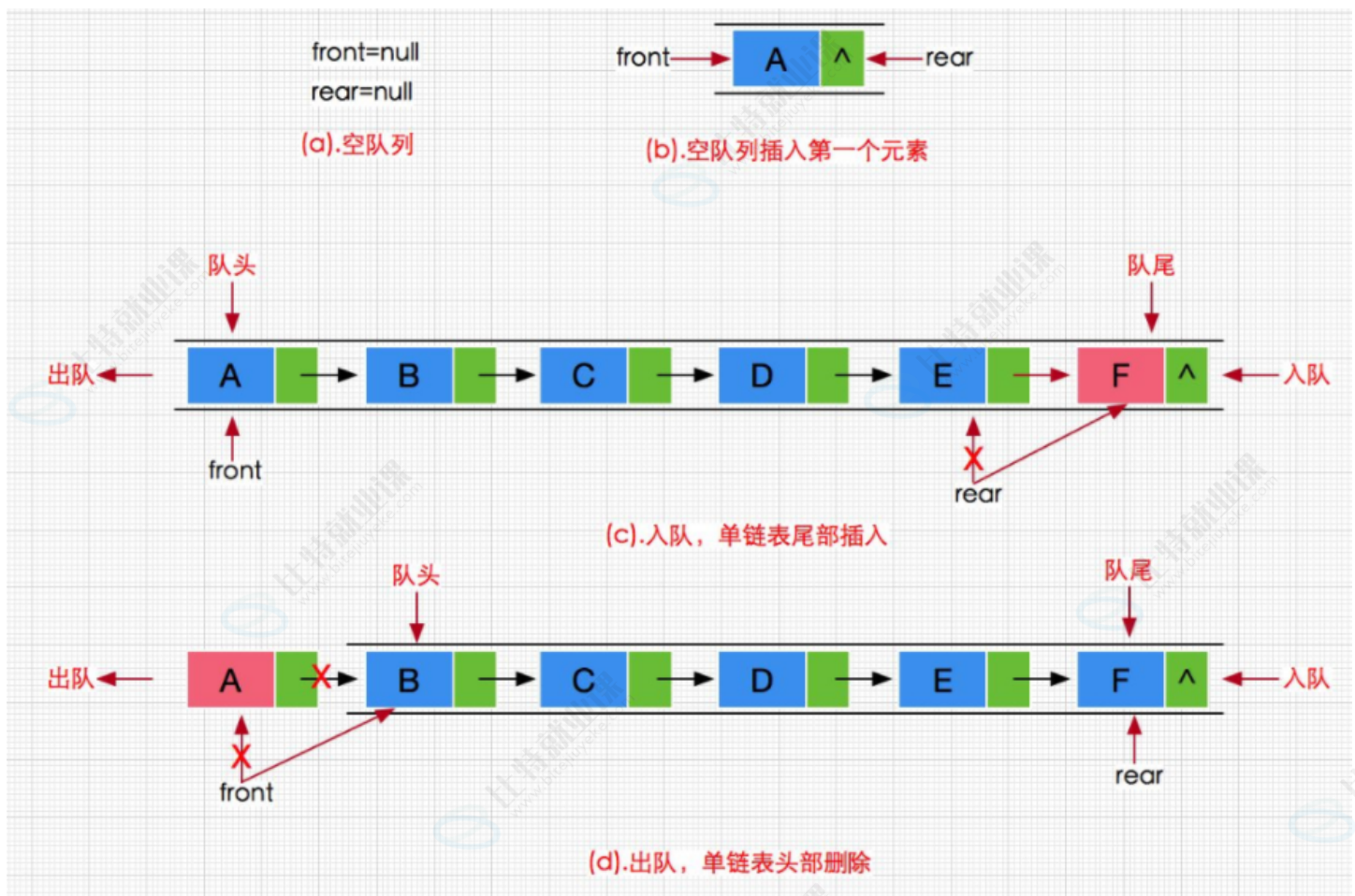
```

5    q.offer(3);
6    q.offer(4);
7    q.offer(5);           // 从队尾入队列
8    System.out.println(q.size());
9    System.out.println(q.peek()); // 获取队头元素
10
11   q.poll();
12   System.out.println(q.poll()); // 从队头出队列，并将删除的元素返回
13
14   if(q.isEmpty()){
15       System.out.println("队列空");
16   }else{
17       System.out.println(q.size());
18   }
19 }

```

2.3 队列模拟实现

队列中既然可以存储元素，那底层肯定要有能够保存元素的空间，通过前面线性表的学习了解到常见的空间类型有两种：**顺序结构**和**链式结构**。同学们思考下：**队列的实现使用顺序结构还是链式结构好？**



```

1 public class Queue {

```



```

2 // 双向链表节点
3 public static class ListNode{
4     ListNode next;
5     ListNode prev;
6     int value;
7
8     ListNode(int value){
9         this.value = value;
10    }
11 }
12
13 ListNode first; // 队头
14 ListNode last; // 队尾
15 int size = 0;
16
17 // 入队列---向双向链表位置插入新节点
18 public void offer(int e){
19     ListNode newNode = new ListNode(e);
20     if(first == null){
21         first = newNode;
22         // last = newNode;
23     }else{
24         last.next = newNode;
25         newNode.prev = last;
26         // last = newNode;
27     }
28
29     last = newNode;
30     size++;
31 }
32
33 // 出队列---将双向链表第一个节点删除掉
34 public int poll(){
35     // 1. 队列为空
36     // 2. 队列中只有一个元素----链表中只有一个节点---直接删除
37     // 3. 队列中有多个元素---链表中多个节点----将第一个节点删除
38     int value = 0;
39     if(first == null){
40         return null;
41     }else if(first == last){
42         last = null;
43         first = null;
44     }else{
45         value = first.value;
46         first = first.next;
47         first.prev.next = null;
48         first.prev = null;

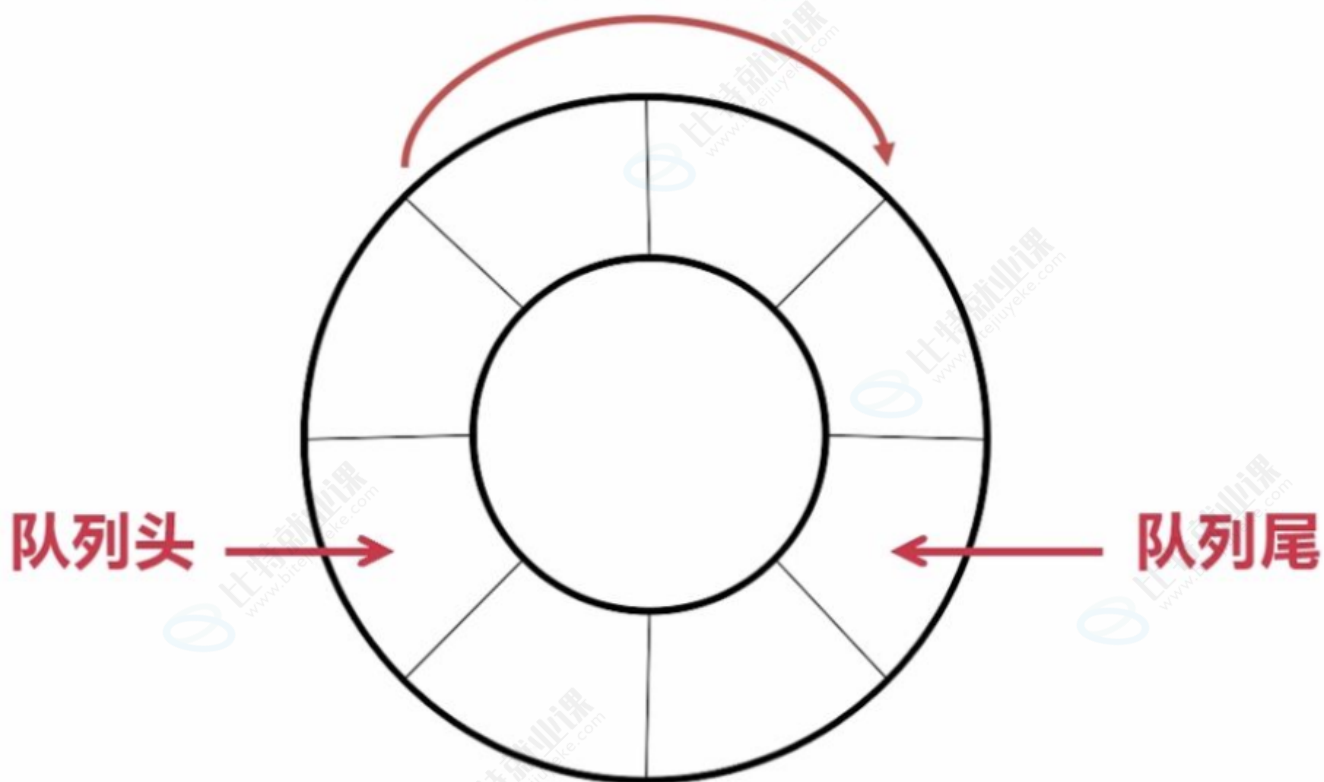
```

```
49     }
50     --size;
51     return value;
52 }
53
54 // 获取队头元素---获取链表中第一个节点的值域
55 public int peek(){
56     if(first == null){
57         return null;
58     }
59
60     return first.value;
61 }
62
63 public int size() {
64     return size;
65 }
66
67 public boolean isEmpty(){
68     return first == null;
69 }
70 }
```

2.4 循环队列

实际中我们有时还会使用一种队列叫循环队列。如操作系统课程讲解生产者消费者模型时就会使用循环队列。环形队列通常使用数组实现。

环形队列



数组下标循环的小技巧

1. 下标最后再往后(offset 小于 array.length): $\text{index} = (\text{index} + \text{offset}) \% \text{array.length}$

array



0 1 2 3 4 5 6 7 8

$(\text{index} + \text{offset}) \% \text{array.length}$

index

index = 7

array.length = 9

offset = 4

结果 = 2

2. 下标最前再往前(offset 小于 array.length): $\text{index} = (\text{index} + \text{array.length} - \text{offset}) \% \text{array.length}$

array



0 1 2 3 4 5 6 7 8

↑ index

↑ $(\text{index} + \text{array.length} - \text{offset}) \% \text{array.length}$

index = 2
array.length = 9
offset = 4
结果 = 7

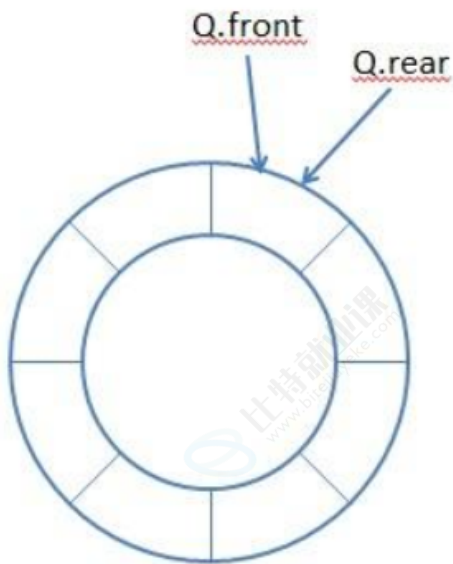
如何区分空与满

1. 通过添加 size 属性记录
2. 保留一个位置
3. 使用标记

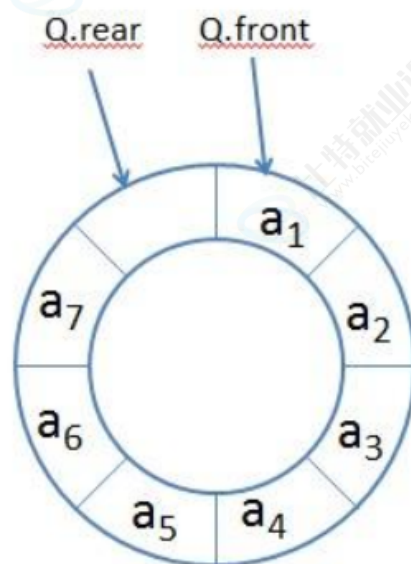
- 初始状态：队列为空时， $\text{front} = \text{rear} = 0$ ，标志位 $\text{isFull} = \text{false}$ 。
- 每次入队检查 rear 是否与 front 重合，如果重合设置 $\text{isFull} = \text{true}$ ，表示队列已满，否则，正常入队并移动 rear
- 出队时，移动 front 指针，设置 $\text{isFull} = \text{false}$ 。

```
1 public boolean enqueue(int value) {  
2     if (isFull) {  
3         return false; // 队列已满  
4     }  
5     array[rear] = value;  
6     rear = (rear + 1) % capacity;  
7     if (rear == front) {  
8         isFull = true;  
9     }  
10    return true;  
11 }  
12  
13 public boolean dequeue() {  
14     if (isEmpty()) {  
15         return false; // 队列为空  
16     }  
17     int item = array[front];  
18     front = (front + 1) % capacity;  
19     isFull = false;  
20    return true;  
}
```

- 判断空: `return !isFull && front == rear;`
- 判断满: `return isFull;`



(a) 空的循环队列



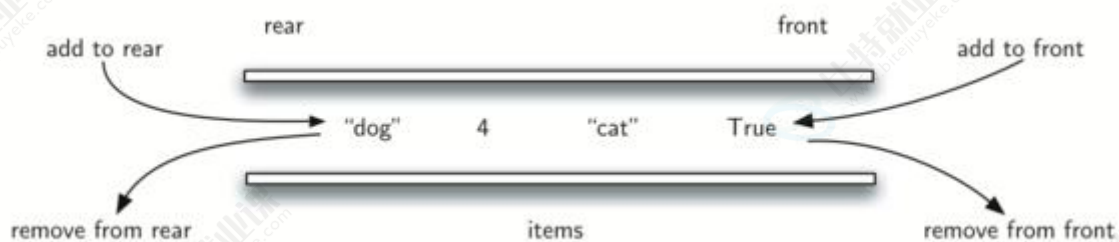
为了使用 $Q.rear = Q.front$ 来区别是队空还是队满，我们常常认为出现左图时的情况即为队满的情况，此时： $rear+1=front$

(b) 满的循环队列

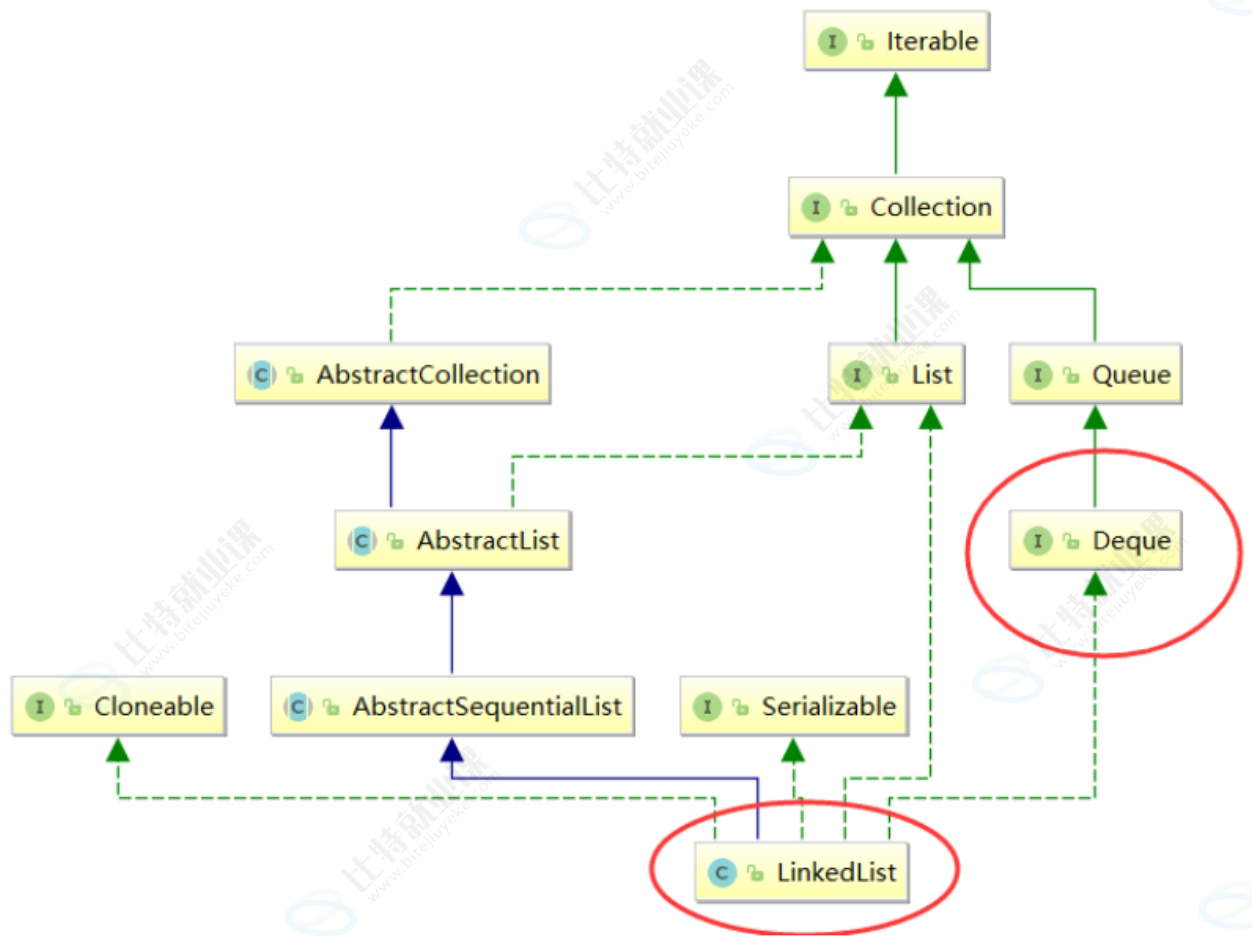
设计循环队列

3. 双端队列 (Deque)

双端队列 (deque) 是指允许两端都可以进行入队和出队操作的队列，deque 是 “double ended queue” 的简称。那就说明元素可以从队头出队和入队，也可以从队尾出队和入队。



Deque是一个接口，使用时必须创建LinkedList的对象。



在实际工程中，使用Deque接口是比较多的，栈和队列均可以使用该接口。

- 1 `Deque<Integer> stack = new ArrayDeque<>();` //双端队列的线性实现
- 2 `Deque<Integer> queue = new LinkedList<>();` //双端队列的链式实现

4. 面试题

1. 用队列实现栈。 [OJ链接](#)
2. 用栈实现队列。 [OJ链接](#)

完